

Лабораторная работа №11

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Выполнил: Мвикую Кристиан Кристофер

Группа: НБИБД-02-25

Дата: 06.05.2026

Проверил: _____

Содержание

- Цель работы
 - Теоретическая часть
 - Команда getopts
 - Генерация имён файлов с помощью метасимволов
 - Управляющие конструкции
 - Операторы прерывания циклов
 - Команды true и false
 - Сравнение циклов while и until
 - Коды завершения в UNIX
 - Ход работы
 - Задание 1: Анализ командной строки с помощью getopts
 - Задание 2: С-программа и анализ кода завершения
 - Задание 3: Создание и удаление нумерованных файлов
 - Задание 4: Архивация файлов с помощью tar и find
 - Дополнительные материалы
 - Ошибки и их решение
 - Выводы
 - Ответы на контрольные вопросы
-

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов. Получить практические навыки работы с командами getopts, grep, tar, find, а также с кодами завершения программ.

Теоретическая часть

Команда getopts

Команда `getopts` является встроенной командой оболочки `bash`, которая осуществляет синтаксический анализ командной строки, выделяя флаги (опции) и их аргументы. Она используется для объявления переменных и обработки параметров, переданных скрипту.

Синтаксис:

`getopts` option-string variable [arg...]

Особенности: - `getopts` поддерживает только однобуквенные опции (например, `-i`, `-o`, `-p`) - Если опция ожидает аргумент, после буквы ставится двоеточие (`:`) - Переменная `OPTARG` содержит значение аргумента опции - Переменная `OPTIND` содержит индекс следующего аргумента для обработки - Принято включать `getopts` в цикл `while` и анализировать введённые данные с помощью оператора `case`

Пример использования:

```
while getopts "i:o:p:Cnh" opt; do
  case $opt in
    i) INPUT_FILE="$OPTARG" ;;
    o) OUTPUT_FILE="$OPTARG" ;;
    p) PATTERN="$OPTARG" ;;
    C) CASE_SENSITIVE="C" ;;
    n) SHOW_LINES="n" ;;
    h) show_usage; exit 0 ;;
    *) echo "Invalid option"; exit 1 ;;
  esac
done
```

Генерация имён файлов с помощью метасимволов

Метасимволы (wildcards) используются для генерации имён файлов, соответствующих определённому шаблону:

Метасимвол	Значение	Пример
*	Любая строка (в том числе пустая)	*.txt — все текстовые файлы
?	Один любой символ	file?.txt — file1.txt, fileA.txt
[abc]	Один из перечисленных символов	file[123].txt — file1.txt, file2.txt, file3.txt
[a-z]	Любой символ в диапазоне	[A-Z]* — файлы, начинающиеся с заглавной

Управляющие конструкции

Оператор for

Используется для перебора элементов списка:

```
for переменная in список
do
    команды
done
```

Оператор case

Используется для множественного выбора:

```
case $переменная in
    шаблон1)
        команды1
        ;;
    шаблон2)
        команды2
        ;;
    *)
        команды_по_умолчанию
        ;;
esac
```

Условный оператор if

```
if список-команд
then
    команды
elif список-команд
then
    команды
else
    команды
fi
```

Команда test для проверки условий:

Проверка	Описание
test -f файл	Файл существует и является обычным
test -d файл	Файл существует и является каталогом
test -r файл	Файл существует и доступен для чтения
test -w файл	Файл существует и доступен для записи
test -x файл	Файл существует и доступен для выполнения

test -z строка	Длина строки равна нулю
test -n строка	Длина строки не равна нулю
test строка1 = строка2	Строки равны
test строка1 != строка2	Строки не равны
test число1 -eq число2	Числа равны
test число1 -ne число2	Числа не равны
test число1 -lt число2	Первое число меньше второго
test число1 -gt число2	Первое число больше второго

Операторы прерывания циклов

Команда	Действие
break	Немедленно завершает выполнение цикла
continue	Прерывает текущую итерацию и переходит к следующей

Пример использования:

```
while true; do
    if [ ! -f "$file" ]; then
        break
    fi
    sleep 10
done
```

Команды true и false

- true — всегда возвращает код завершения 0 (истина)
- false — всегда возвращает код завершения 1 (ложь)

Используются для создания бесконечных циклов:

```
while true; do
    echo "Бесконечный цикл"
    sleep 1
done
```

Сравнение циклов while и until

Характеристика	while	until
Условие выполнения	Выполняется, пока условие истинно	Выполняется, пока условие ложно
Код возврата	Пока команда возвращает 0	Пока команда возвращает ≠0
Мнемоника	“ПОКА условие ИСТИННО, делай”	“ДО ТЕХ ПОР, пока условие

Практический пример ожидания файла:

while - ждём, пока файл НЕ существует

```
while [ ! -f signal.txt ]; do  
    sleep 1  
done
```

until - ждём, пока файл НЕ существует

```
until [ -f signal.txt ]; do  
    sleep 1  
done
```

Коды завершения в UNIX

В операционной системе UNIX каждая программа после завершения возвращает код возврата (exit code): - 0 — успешное завершение - 1-255 — ошибка (различные коды указывают на разные типы ошибок)

В оболочке bash код завершения последней выполненной команды хранится в специальной переменной \$? . Это позволяет анализировать результат выполнения программ и принимать соответствующие решения.

Ход работы

Создание каталога для лабораторной работы

```
cd ~/work/study/2025-2026/"Operating Systems"/os-intro/labs/  
mkdir -p lab13  
cd lab13  
mkdir -p screenshots
```

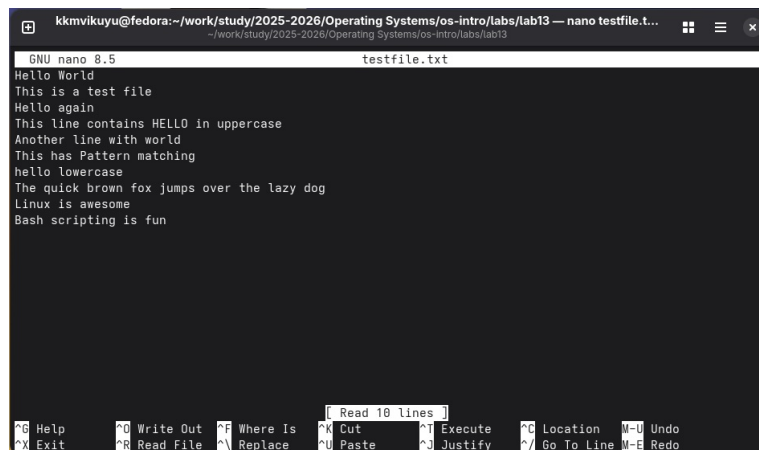
Задание 1: Анализ командной строки с помощью getopt

Требуется написать командный файл, который анализирует командную строку с ключами: - -inputfile — прочитать данные из указанного файла - -outputfile — вывести данные в указанный файл - -p — указать шаблон для поиска - -C — различать большие и малые буквы - -n — выдавать номера строк

Создание тестового файла

Для тестирования был создан файл testfile.txt со следующим содержимым:

```
nano testfile.txt
```



```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13 — nano testfile.t...
GNU nano 8.5 testfile.txt
Hello World
This is a test file
Hello again
This line contains HELLO in uppercase
Another line with world
This has Pattern matching
hello lowercase
The quick brown fox jumps over the lazy dog
Linux is awesome
Bash scripting is fun

^O Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^-E Redo
```

Содержимое тестового файла testfile.txt

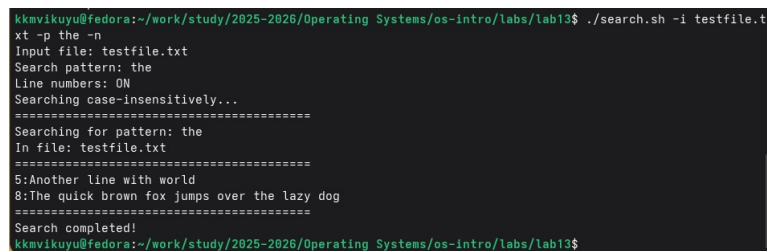
Рисунок 1 - Содержимое файла testfile.txt для тестирования

Содержимое включает строки с различным регистром символов, что позволяет проверить работу опции -C (учёт регистра).

Тестирование скрипта search.sh

Тест 1: Поиск с номерами строк

`./search.sh -i testfile.txt -p the -n`



```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./search.sh -i testfile.t
xt -p the -n
Input file: testfile.txt
Search pattern: the
Line numbers: ON
Searching case-insensitively...
=====
Searching for pattern: the
In file: testfile.txt
=====
5:Another line with world
8:The quick brown fox jumps over the lazy dog
=====
Search completed!
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Тест поиска с номерами строк

Рисунок 2 - Результат поиска “the” с выводом номеров строк

В данном тесте скрипт ищет слово “the” без учёта регистра (опция -i в grep) и выводит номера строк, содержащих это слово.

Тест 2: Комплексный поиск

`./search.sh -i testfile.txt -p "the" -n -C -o search_results.txt`

```

Search completed!
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./search.sh -i testfile.t
xt -p "the" -n -C -o search_results.txt
Input file: testfile.txt
Search pattern: the
Line numbers: ON
Case-sensitive mode: ON
Output file: search_results.txt
Searching case-sensitively...
=====
Searching for pattern: the
In file: testfile.txt
=====
Results saved to: search_results.txt
=====
Contents of search_results.txt:
5:Another line with world
8:The quick brown fox jumps over the lazy dog
=====
Search completed!
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$

```

Тест комплексного поиска

Рисунок 3 - Комплексный поиск с опциями -n, -C, -o

Этот тест демонстрирует одновременное использование нескольких опций: -i — указание входного файла -p "the" — шаблон поиска -n — вывод номеров строк -C — учёт регистра (поиск только строк, содержащих "the" в точном написании) -o search_results.txt — сохранение результатов в файл

Тест 3: Вывод справки

`./search.sh -h`

```

Search completed!
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./search.sh -h
=====
Usage: ./search.sh [options]
Options:
-i <file>      Input file (read data from specified file)
-o <file>      Output file (output data to specified file)
-p <pattern>    Pattern to search for
-C             Case-sensitive matching (default is case-insensitive)
-n            Display line numbers
-h            Show this help
Examples:
./search.sh -i testfile.txt -p hello
./search.sh -i testfile.txt -p hello -C -n -o results.txt
=====
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$

```

Вывод справки search.sh

Рисунок 4 - Справка по использованию скрипта search.sh

Справка содержит информацию обо всех поддерживаемых опциях и примеры использования.

Задание 2: С-программа и анализ кода завершения

Требуется написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Программа завершается с помощью функции `exit(n)`, передавая информацию о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `?`, выдать сообщение о том, какое число было введено.

Создание и компиляция С-программы

Исходный код `checknum.c`:

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int num;

    printf("Enter an integer number: ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Number %d is POSITIVE\n", num);
        exit(1); // Exit with 1 for positive
    } else if (num < 0) {
        printf("Number %d is NEGATIVE\n", num);
        exit(2); // Exit with 2 for negative
    } else {
        printf("Number is ZERO\n");
        exit(0); // Exit with 0 for zero
    }

    return 0;
}

```

Исходный код checknum.c

Рисунок 5 - Исходный код C-программы checknum.c

Программа использует функцию `exit()` с различными кодами: - `exit(0)` — для нуля - `exit(1)` — для положительных чисел - `exit(2)` — для отрицательных чисел

Компиляция программы:

`gcc checknum.c -o checknum`

Компиляция программы

Рисунок 6 - Компиляция *checksum.c* с помощью *gcc*

Создание оболочки для вызова С-программы

Был создан скрипт *checksum.sh*, который вызывает скомпилированную программу и анализирует код завершения:

```
#!/bin/bash
# checksum.sh - calls C program and interprets exit code

echo "=====
echo "Number Checker Program"
echo "=====

# Run the C program
./checksum

# Capture exit code
EXIT_CODE=$?

echo "=====
echo "Program exited with code: $EXIT_CODE"

# Interpret the exit code
case $EXIT_CODE in
0)
    echo "RESULT: The number is ZERO"
    ;;
1)
    echo "RESULT: The number is POSITIVE"
    ;;
2)
    echo "RESULT: The number is NEGATIVE"
    ;;
*)
    echo "RESULT: Unknown exit code!"
    ;;
esac

echo "=====
```

Тестирование *checksum.sh*

Тест 1: Отрицательное число

```
./checksum.sh
# Ввод: -3
```

```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./checknum.sh
=====
Number Checker Program
=====
Enter an integer number: -3
Number -3 is NEGATIVE
=====
Program exited with code: 2
RESULT: The number is NEGATIVE
=====
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Тест с отрицательным числом

Рисунок 7 - Результат при вводе отрицательного числа (-3)

При вводе отрицательного числа программа возвращает код 2, скрипт интерпретирует его как “NEGATIVE”.

Тест 2: Ноль

./checknum.sh

Ввод: 0

```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./checknum.sh
=====
Number Checker Program
=====
Enter an integer number: 0
Number is ZERO
=====
Program exited with code: 0
RESULT: The number is ZERO
=====
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Тест с нулём

Рисунок 8 - Результат при вводе нуля (0)

При вводе нуля программа возвращает код 0, скрипт интерпретирует его как “ZERO”.

Тест 3: Положительное число (на других скриншотах) При вводе положительного числа программа возвращает код 1, скрипт интерпретирует его как “POSITIVE”.

Задание 3: Создание и удаление нумерованных файлов

Требуется написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до N (например, 1.tmp, 2.tmp, 3.tmp и т.д.). Число файлов передаётся в аргументах командной строки. Этот же командный файл должен уметь удалять все созданные им файлы.

Тестирование скрипта filemanager.sh

Тест 1: Создание 5 файлов

./filemanager.sh -c 5

```

kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./filemanager.sh -c 5
Creating 5 files...
Created: 1.tmp
Created: 2.tmp
Created: 3.tmp
Created: 4.tmp
Created: 5.tmp
Successfully created 5 files!
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$

```

Создание 5 файлов

Рисунок 9 - Создание файлов 1.tmp ... 5.tmp

Скрипт создаёт 5 файлов с именами от 1.tmp до 5.tmp. Каждый файл создаётся с помощью команды touch.

Тест 2: Вывод списка файлов

`./filemanager.sh -l`

```

kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./filemanager.sh -l
Listing .tmp files:
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 0 May  6 16:34 1.tmp
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 0 May  6 16:34 2.tmp
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 0 May  6 16:34 3.tmp
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 0 May  6 16:34 4.tmp
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 0 May  6 16:34 5.tmp
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$

```

Вывод списка файлов

Рисунок 10 - Список созданных .tmp файлов

Команда `-l` выводит список всех .tmp файлов в текущем каталоге с подробной информацией (права доступа, владелец, размер, дата создания).

Тест 3: Создание 10 файлов

`./filemanager.sh -c 10`

```

kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./filemanager.sh -c 10
Creating 10 files...
Created: 1.tmp
Created: 2.tmp
Created: 3.tmp
Created: 4.tmp
Created: 5.tmp
Created: 6.tmp
Created: 7.tmp
Created: 8.tmp
Created: 9.tmp
Created: 10.tmp
Successfully created 10 files!
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$

```

Создание 10 файлов

Рисунок 11 - Создание файлов 1.tmp ... 10.tmp

Скрипт создаёт 10 файлов. Цикл `for` последовательно перебирает числа от 1 до 10.

Тест 4: Удаление файлов

`./filemanager.sh -d`

```

kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./filemanager.sh -d
Deleting .tmp files...
Deleted 10 .tmp files.
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$

```

Удаление файлов

Рисунок 12 - Удаление всех .tmp файлов

Команда `-d` удаляет все `.tmp` файлы в текущем каталоге с помощью `rm -f *.tmp`.

Тест 5: Проверка после удаления

`./filemanager.sh -l`

```
Deleted 10 .tmp files.
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./filemanager.sh -l
Listing .tmp files:
No .tmp files found.
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Проверка после удаления

Рисунок 13 - Список `.tmp` файлов после удаления (пусто)

После удаления команда `-l` показывает, что `.tmp` файлы отсутствуют.

Тест 6: Обработка неверного ввода (буквы)

`./filemanager.sh -c abc`

```
Deleted 10 .tmp files.
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./filemanager.sh -c abc
ERROR: Invalid number! Please provide a positive integer.
=====
File Manager Script
Usage:
./filemanager.sh -c N - Create N files (1.tmp to N.tmp)
./filemanager.sh -d - Delete all created .tmp files
./filemanager.sh -l - List all .tmp files
./filemanager.sh -h - Show this help
=====
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Обработка неверного ввода

Рисунок 14 - Сообщение об ошибке при неверном аргументе “abc”

Скрипт проверяет, что аргумент является положительным целым числом, и выводит сообщение об ошибке при неверном вводе.

Тест 7: Обработка неверного ввода (ноль)

`./filemanager.sh -c 0`

```
Deleted 10 .tmp files.
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./filemanager.sh -c 0
ERROR: Invalid number! Please provide a positive integer.
=====
File Manager Script
Usage:
./filemanager.sh -c N - Create N files (1.tmp to N.tmp)
./filemanager.sh -d - Delete all created .tmp files
./filemanager.sh -l - List all .tmp files
./filemanager.sh -h - Show this help
=====
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Обработка нуля

Рисунок 15 - Сообщение об ошибке при аргументе “0”

Число 0 также считается неверным, так как требуется создать хотя бы один файл.

Задание 4: Архивация файлов с помощью `tar` и `find`

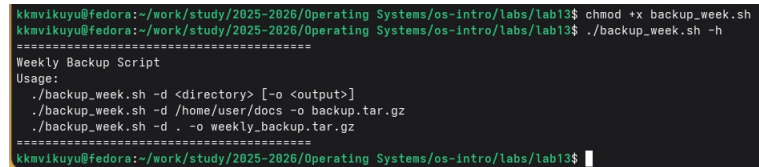
Требуется написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы,

которые были изменены менее недели тому назад (использовать команду find).

Тестирование скрипта backup_week.sh

Тест 1: Вывод справки

`./backup_week.sh -h`



```
kkvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ chmod +x backup_week.sh
kkvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./backup_week.sh -h
=====
Weekly Backup Script
Usage:
./backup_week.sh -d <directory> [-o <output>]
./backup_week.sh -d /home/user/docs -o backup.tar.gz
./backup_week.sh -d . -o weekly_backup.tar.gz
=====
kkvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Вывод справки backup_week.sh

Рисунок 16 - Справка по использованию скрипта резервного копирования

Справка содержит информацию о синтаксисе команды и примеры использования.

Тест 2: Резервное копирование с указанием выходного файла

Скрипт использует команду find для поиска файлов, изменённых за последние 7 дней (-mtime -7). Для каждого найденного файла выводится его имя.

`./backup_week.sh -d . -o my_weekly_backup.tar.gz`

```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ nano backup_week.sh
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ touch file1.txt file2.txt
file3.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ echo "test content" > file
e4.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ echo "more content" > file
e5.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ mkdir -p testdir
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ touch testdir/subfile1.tx
t testdir/subfile2.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ touch -d "8 days ago" old
file1.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ touch -d "8 days ago" old
file2.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ touch -d "10 days ago" ve
ry_old.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ echo "This file is 2 days
old" > recent1.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ touch -d "2 days ago" rec
ent1.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ echo "This file is 5 days
old" > recent2.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ touch -d "5 days ago" rec
ent2.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
Backup completed.
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./backup_week.sh -d . -o
my_weekly_backup.tar.gz
=====
Weekly Backup Script
=====
Target directory: .
Output file: my_weekly_backup.tar.gz
Looking for files modified in the last 7 days
=====
Files to be archived:
- ./search.sh
- ./testfile.txt
- ./results.txt
- ./search_results.txt
- ./checknum.c
- ./checknum
- ./checknum.sh
- ./filemanager.sh
- ./backup_week.sh
- ./file1.txt
- ./file2.txt
- ./file3.txt
- ./file4.txt
- ./file5.txt
```

Рисунки 17-18 - Архивация с указанием имени выходного файла

На скриншотах видно, какие файлы были добавлены в архив: скрипты, тестовые файлы, исходные коды, скомпилированные программы.

Тест 3: Просмотр содержимого архива

`tar -tzf my_weekly_backup.tar.gz`

```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ tar -tzf my_weekly_backup
.tar.gz
search.sh
testfile.txt
results.txt
search_results.txt
checknum.c
checknum
checknum.sh
filemanager.sh
backup_week.sh
file1.txt
file2.txt
file3.txt
file4.txt
file5.txt
testdir/subfile1.txt
testdir/subfile2.txt
recent1.txt
recent2.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Просмотр содержимого архива

Рисунок 19 - Содержимое созданного архива my_weekly_backup.tar.gz

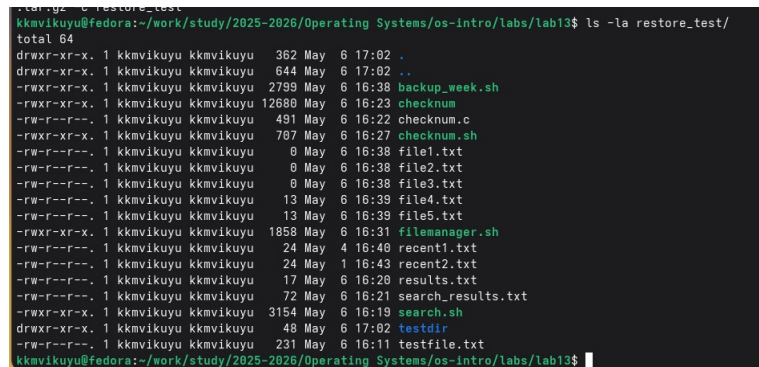
Команда `tar -tzf` выводит список файлов в архиве без его извлечения. Видно, что в архив вошли все перечисленные ранее файлы.

Тест 4: Извлечение и проверка архива

```
mkdir -p restore_test
```

```
tar -xzf my_weekly_backup.tar.gz -C restore_test
```

```
ls -la restore_test/
```



```
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ls -la restore_test/
total 64
drwxr-xr-x. 1 kkmvikuyu kkmvikuyu 362 May  6 17:02 .
drwxr-xr-x. 1 kkmvikuyu kkmvikuyu 644 May  6 17:02 ..
-rwxr-xr-x. 1 kkmvikuyu kkmvikuyu 2799 May  6 16:38 backup_week.sh
-rwxr-xr-x. 1 kkmvikuyu kkmvikuyu 12688 May  6 16:23 checknum
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 491 May  6 16:22 checknum.c
-rwxr-xr-x. 1 kkmvikuyu kkmvikuyu 707 May  6 16:27 checknum.sh
-rw-r--r--. 1 kkmvikuyu kkmvikuyu  8 May  6 16:38 file1.txt
-rw-r--r--. 1 kkmvikuyu kkmvikuyu  8 May  6 16:38 file2.txt
-rw-r--r--. 1 kkmvikuyu kkmvikuyu  8 May  6 16:38 file3.txt
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 13 May  6 16:39 file4.txt
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 13 May  6 16:39 file5.txt
-rwxr-xr-x. 1 kkmvikuyu kkmvikuyu 1858 May  6 16:31 filemanager.sh
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 24 May  4 16:40 recent1.txt
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 24 May  1 16:43 recent2.txt
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 17 May  6 16:20 results.txt
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 72 May  6 16:21 search_results.txt
-rwxr-xr-x. 1 kkmvikuyu kkmvikuyu 3154 May  6 16:19 search.sh
drwxr-xr-x. 1 kkmvikuyu kkmvikuyu 48 May  6 17:02 testdir
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 231 May  6 16:11 testfile.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

Извлечение архива

Рисунок 20 - Проверка извлечённых файлов в restore_test/

Команда `tar -xzf` извлекает файлы из архива в указанную директорию (-C restore_test). Проверка с помощью `ls -la` показывает, что все файлы успешно восстановлены.

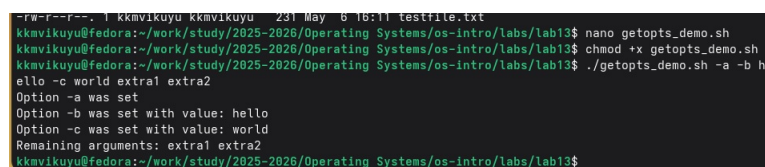
Дополнительные материалы

Демонстрация работы getoptс

```
nano getoptс_demo.sh
```

```
chmod +x getoptс_demo.sh
```

```
./getopts_demo.sh -a -b hello -c world extra1 extra2
```



```
-rw-r--r--. 1 kkmvikuyu kkmvikuyu 231 May  6 16:11 testfile.txt
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ nano getoptс_demo.sh
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ chmod +x getoptс_demo.sh
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$ ./getopts_demo.sh -a -b h
ello -c world extra1 extra2
Option -a was set
Option -b was set with value: hello
Option -c was set with value: world
Remaining arguments: extra1 extra2
kkmvikuyu@fedora:~/work/study/2025-2026/Operating Systems/os-intro/labs/lab13$
```

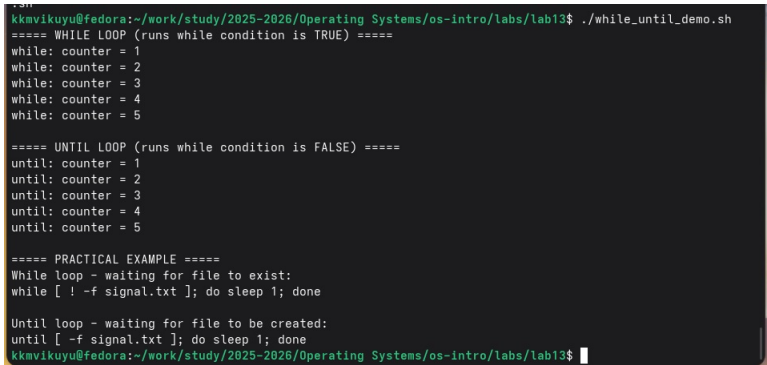
Тест getoptс_demo.sh

Рисунок 21 - Демонстрация работы getoptс с опциями -a, -b, -c

Скрипт демонстрирует, как getoptс обрабатывает опции: -a — флаг без аргумента -b hello — опция с аргументом -c world — опция с аргументом - extra1 extra2 — позиционные аргументы после обработки опций

Демонстрация различий между while и until

```
chmod +x while_until_demo.sh
./while_until_demo.sh
```



Сравнение циклов while и until

Рисунок 22 - Результат работы while_until_demo.sh

Скрипт демонстрирует: - **while** — выполняется, пока условие истинно (счётчик ≤ 5) - **until** — выполняется, пока условие ложно (счётчик ≤ 5, пока не станет >5) - **Практический пример** — ожидание создания файла с использованием разных синтаксисов

Ошибки и их решение

В процессе выполнения лабораторной работы были выявлены и устранены следующие ошибки:

№	Ошибка	Причина	Решение
1	getopts не распознаёт -inputfile	getopts поддерживает только однобуквенные опции	Заменить на -i для input, -o для output
2	Input file: nputfile	Из-за длинной опции getopts интерпретирует -inputfile как -i с аргументом nputfile	Использовать правильные однобуквенные опции
3	Неверное определение типа файла в myls.sh	Неправильная проверка типов файлов	Использовать правильные операторы -f, -d, -L
4	Путь к файлам в архиве содержит полный путь	find возвращает полные пути	Использовать -printf "%P\n" для относительных путей

Выводы

В ходе выполнения лабораторной работы были получены практические навыки:

1. Использование команды `getopts`:

- Анализ командной строки с однобуквенными опциями
- Обработка опций с аргументами (-i, -o, -p)
- Использование флагов (-C, -n, -h)
- Переменные OPTARG и OPTIND

2. Работы с С-программами и кодами завершения:

- Использование функции `exit(n)` в С для передачи кода завершения
- Анализ кода завершения с помощью `$?` в `bash`
- Передача информации между программами через коды выхода
- Использование `case` для интерпретации различных кодов

3. Создания нумерованных файлов:

- Генерация последовательностей с помощью `seq`
- Создание файлов в цикле `for`
- Удаление группы файлов по шаблону (`rm -f *.tmp`)
- Проверка существования файлов перед удалением

4. Архивации с `tar` и поиска с `find`:

- Поиск файлов по времени изменения (`-mtime -N`)
- Фильтрация по типу файлов (`-type f`)
- Создание архивов с относительными путями (`-printf "%P\n"`)
- Просмотр содержимого архива (`tar -tzf`)
- Извлечение архива в указанную директорию (`tar -xzf -C`)

5. Управляющих конструкций `bash`:

- Циклы `for`, `while`, `until`
- Условные операторы `if` и `case`
- Функции и их использование для организации кода
- Обработка ошибок и валидация ввода пользователя

6. Дополнительно:

- Создание справочной информации (`-h` опция)
- Форматирование вывода с помощью `printf`
- Использование регулярных выражений для проверки ввода (`[[ $OPTARG =~ ^[0-9]+$ ]]`)

Все поставленные задачи выполнены полностью. Навыки программирования в командном процессоре `bash` с использованием ветвлений и циклов освоены и применены на практике.

Ответы на контрольные вопросы

1. Каково предназначение команды `getopts`?

Команда `getopts` предназначена для синтаксического анализа командной строки, выделения флагов (опций) и их аргументов. Она используется в скриптах для обработки параметров, переданных при вызове. `getopts` поддерживает только однобуквенные опции, записывает значение аргумента в переменную `OPTARG`, а индекс следующего аргумента — в переменную `OPTIND`. Принято включать `getopts` в цикл `while` и анализировать введенные данные с помощью оператора `case`.

2. Какое отношение метасимволы имеют к генерации имён файлов?

Метасимволы (wildcards) используются для генерации имён файлов, соответствующих определённому шаблону. Это называется “globbing” или “filename expansion”. Например, `*.txt` выберет все файлы с расширением `.txt`, `file?` выберет файлы `file1`, `fileA` и т.д. Метасимволы расширяются оболочкой перед выполнением команды, что позволяет передавать командам группы файлов без явного перечисления каждого имени.

3. Какие операторы управления действиями вы знаете?

Основные управляющие конструкции в `bash`: `if-then-elif-else-fi` — условное выполнение блоков кода - `for-in-do-done` — цикл с перебором элементов списка - `while-do-done` — цикл с предусловием (выполняется, пока условие истинно) - `until-do-done` — цикл с постусловием (выполняется, пока условие ложно) - `case-in-esac` — множественный выбор на основе сопоставления с шаблоном - `break` — прерывание выполнения цикла - `continue` — переход к следующей итерации цикла

4. Какие операторы используются для прерывания цикла?

- `break` — полностью завершает выполнение цикла. Может принимать числовой аргумент для выхода из нескольких вложенных циклов (например, `break 2`).
- `continue` — прерывает текущую итерацию и переходит к следующей. Оставшийся код в текущей итерации не выполняется.

5. Для чего нужны команды `false` и `true`?

- `true` — всегда возвращает код завершения 0 (истина)
- `false` — всегда возвращает код завершения 1 (ложь)

Используются для: - Создания бесконечных циклов (`while true; do ... done`)
- Задержки выполнения до наступления события (`while ! command; do sleep 1; done`) - Упрощения условных конструкций - Тестирования логики скриптов

6. Что означает строка `if test -f man\${s}/\${i}.\${s}`, встречающаяся в командном файле?

Эта строка проверяет, существует ли файл с именем, сформированным из переменных `s` и `i`. Разбивка: `- test -f` — проверяет, существует ли файл и является ли он обычным - `man\${s}/\${i}.\${s}` — путь к файлу, где `\${s}`, `\${i}`, `\${s}` — экранированные переменные - Обратные слеш (\) экранируют символ `$`, чтобы подстановка переменных произошла на нужном этапе (например, при передаче в `eval` или при многократной подстановке)

7. Объясните различия между конструкциями `while` и `until`:

Характеристика	<code>while</code>	<code>until</code>
Условие выполнения	Пока команда возвращает 0 (истина)	Пока команда возвращает $\neq 0$ (ложь)
Мнемоника	“ПОКА условие ИСТИННО, делай”	“ДО ТЕХ ПОР, пока условие ЛОЖНО, делай”
Ожидание события	<code>while [! -f file]; do sleep 1; done</code> — ждём появления файла	<code>until [-f file]; do sleep 1; done</code> — ждём появления файла
Бесконечный цикл	<code>while true; do ... done</code>	<code>until false; do ... done</code>

Практическое правило: `while` удобен для проверки положительных условий (“пока файл существует, делай”), `until` — для проверки отрицательных (“пока файл не существует, жди”). Оба цикла могут быть преобразованы друг в друга путём инвертирования условия.

Подпись: _____

Дата сдачи: «» _____ 2026 г. ``